

STREAMLIT 2026



CHULETA EXTENDIDA DE CERO A PROFESIONAL

Ayuda memoria visual para crear aplicaciones web con Python

01

Entender

02

Construir

03

Depurar

04

Publicar

EDICIÓN AMPLIADA

Sintaxis actual | ejemplos listos para copiar | decisiones | recetas | errores frecuentes

Documento para estudiantes



Cómo usar esta chuleta

Consulta rápida primero; profundidad cuando la necesites

STREAMLIT FIELD GUIDE

Edición ampliada 2026

1

BUSCA POR COLOR

Coral: interfaz. Azul: datos. Verde: estado y rendimiento.
Violeta: arquitectura.

2

COPIA EL MÍNIMO

Proba primero el ejemplo corto. Luego cambia un valor por vez y observa el rerun.

3

LEE LA PISTA

Cada tarjeta dice para que sirve, muestra una receta y anticipa un error habitual.

4

SUBI DE NIVEL

Inicio -> widgets -> datos -> estado -> multipágina -> pruebas -> despliegue.



Mapa de aprendizaje

Una ruta sugerida; podes consultar cualquier ficha de forma independiente

STREAMLIT FIELD GUIDE

Edición ampliada 2026

01

Mostrar

texto, Markdown, mensajes



02

Preguntar

widgets, botones, formularios



03

Ordenar

layout, tablas, gráficos



04

Recordar

estado, callbacks, cache



05

Escalar

páginas, datos, pruebas



06

Publicar

seguridad, accesibilidad, nube





Índice de contenidos I

Ubica el tema y salta directamente a la ficha que necesitas

STREAMLIT FIELD GUIDE

Edición ampliada 2026

06

Arranque ultrarrápido

07

El modelo mental correcto

08

Configurar la página

09

Texto esencial

10

Markdown que funciona

11

Mensajes y feedback

12

Botones y acciones

13

Entradas de texto

14

Números, fechas y horas

15

Elegir una opción

16

Elegir varias opciones

17

Color y calificación

18

Archivos, cámara y audio

19

Formularios

20

Keys y callbacks

21

Contenedores

22

Columnas, tabs y sidebar

23

Tablas y DataFrames

24

Editor de datos

25

Column config

26

Métricas, JSON y objetos

27

Gráficos nativos

28

Altair y Plotly

29

Imagen, audio y video

30

Descargas y exportación

31

Session State: fundamentos

32

Patrones de estado



Índice de contenidos II

Ubica el tema y salta directamente a la ficha que necesitas

STREAMLIT FIELD GUIDE

Edición ampliada 2026

33

Control del flujo

34

Cache sin misterios

35

Fragments

36

Dialogos, progreso y tareas

37

Apps multipágina

38

URL, contexto y navegador

39

Temas y configuración

40

Secrets, conexiones y APIs

41

Persistencia con SQLite

42

Chat e IA

43

Pruebas con AppTest

44

Depuracion profesional

45

Despliegue

46

Arquitectura del proyecto

47

Rendimiento

48

Seguridad y privacidad

49

Accesibilidad y UX

50

Receta: buscador de datos

51

Receta: quiz con memoria

52

Receta: dashboard

53

Clinica de errores I

54

Clinica de errores II

55

Que componente uso

56

Checklist antes de entregar



Arranque ultrarrapido

De archivo vacío a app funcionando

STREAMLIT FIELD GUIDE

Edición ampliada 2026

INICIO La app mínima

Streamlit ejecuta el archivo de arriba hacia abajo y convierte cada comando `st.*` en interfaz.

```
import streamlit as st
st.write("Hola, Streamlit")
```

- Guarda como `app.py`.

TERMINAL Ejecutar localmente

Instala una vez y ejecuta desde la carpeta del proyecto.
El navegador suele abrir `localhost:8501`.

```
python -m pip install streamlit
streamlit run app.py
```

- Detenela con **Ctrl+C**.

WEB Probar sin instalar

Para ejercicios iniciales usa el Playground oficial.
Para proyectos en navegador, Stlite o Py.Cafe pueden ser útiles.

```
https://streamlit.io/playground
```

- Los entornos web pueden tener límites de paquetes o archivos.

DEPENDENCIAS requirements.txt

Anota las bibliotecas necesarias, una por línea. Esto permite reproducir y desplegar la app.

```
streamlit
pandas
plotly
requests
```

- Fija versiones solo cuando necesites reproducibilidad exacta.



El modelo mental correcto

Por qué la app se vuelve a ejecutar y como aprovecharlo

STREAMLIT FIELD GUIDE

Edición ampliada 2026

FLUJO Rerun completo

Cuando cambia un widget, Streamlit normalmente vuelve a ejecutar el script completo de arriba hacia abajo.

```
nombre = st.text_input("Nombre")
st.write("Hola", nombre)
```

- No imagines una página HTML estática: imagina un guion que se recalcula.

WIDGETS El valor es el retorno

La función del widget devuelve el valor actual. Guárdalo en una variable y usalo después.

```
edad = st.slider("Edad", 0, 100, 18)
st.write(edad)
```

- Casi todos los widgets siguen este patrón.

EVENTO El botón es momentáneo

`st.button` devuelve `True` solamente en el rerun disparado por el clic.

```
if st.button("Celebrar"):
    st.balloons()
```

- Para recordar un cambio, usa `st.session_state`.

LECTURA Orden de aparición

Los elementos aparecen siguiendo el orden del código, salvo que los coloques dentro de contenedores.

```
st.title("A")
st.write("B")
st.caption("C")
```

- Lee el archivo como una historia visual.



Configurar la página

Título del navegador, icono, ancho y menú

STREAMLIT FIELD GUIDE

Edición ampliada 2026

PÁGINA `st.set_page_config`

Debe ejecutarse antes de otros comandos Streamlit en cada página.

```
st.set_page_config(
    page_title="Mi app",
    page_icon="📄",
    layout="wide"
)
```

- Usa `layout='centered'` para lectura y `'wide'` para dashboards.

PÁGINA Estado inicial de sidebar

Controla si la barra lateral comienza abierta, cerrada o automática.

```
st.set_page_config(
    initial_sidebar_state="expanded"
)
```

- En celular Streamlit adapta el layout.

PÁGINA Menu del navegador

`menu_items` permite agregar ayuda, reporte y descripción Acerca de.

```
st.set_page_config(menu_items={
    "Get Help": "https://...",
    "About": "Proyecto escolar"
})
```

- Los enlaces deben ser públicos y claros.

MARCA Logo de la app

Muestra una marca en la zona superior y puede usar una imagen distinta como icono compacto.

```
st.logo("logo.png",
    icon_image="icono.png")
```

- Prueba contraste en tema claro y oscuro.



TEXTO Titulos

Construye una jerarquía real. Un solo título principal y secciones ordenadas.

```
st.title("Mi proyecto")
st.header("Resultados")
st.subheader("Detalle")
```

- No uses tamaños solo por estética: expresan estructura.

COMODIN `st.write`

Muestra texto, números, listas, diccionarios, DataFrames, gráficos y muchos objetos.

```
st.write("Total:", 42)
st.write({"nivel": 3})
```

- Es la salida más versátil para empezar.

LITERAL `st.text` y `st.code`

`st.text` conserva un estilo simple; `st.code` aplica formato monoespaciado y resaltado.

```
st.text("Salida literal")
st.code("x = 2 + 2", language="python")
```

- Para código largo, usa `st.code`.

DETALLE `st.caption` y `st.divider`

La `caption` agrega contexto secundario. `divider` separa bloques visuales.

```
st.caption("Actualizado hoy")
st.divider()
```

- El texto secundario no debe contener lo imprescindible.



MARKDOWN

Negrita, cursiva y listas

Usa Markdown para estructurar explicaciones, consignas y resultados.

```
st.markdown("**Importante**")  
st.markdown("- Uno\n- Dos")
```

- Separa párrafos con una línea vacía.

MARKDOWN

Enlaces y citas

Crea enlaces descriptivos y citas breves.

```
st.markdown("[Documentacion](https://docs.streamlit.io)")  
st.markdown("> Una idea clave")
```

- Evita 'hace clic aquí' como único texto del enlace.

LATEX

Matematica

st.latex renderiza expresiones; st.markdown también admite fragmentos matemáticos.

```
st.latex(r"E = mc^2")  
st.markdown(r"La media es  $\bar{x}$ ")
```

- Prefiere fórmulas solo cuando ayuden a comprender.

SEGURIDAD

HTML: con cuidado

unsafe_allow_html permite HTML dentro de Markdown, pero aumenta riesgos y mantenimiento.

```
st.markdown("<b>Hola</b>",  
            unsafe_allow_html=True)
```

- No insertes contenido HTML recibido de usuarios.



FEEDBACK

Estados semánticos

Usa colores con significado consistente: éxito, información, advertencia y error.

```
st.success("Guardado")
st.info("Dato opcional")
st.warning("Revisa")
st.error("No se pudo")
```

- El mensaje debe explicar la siguiente acción.

DEBUG

Excepciones

st.exception muestra una excepcion de Python con su traza, útil durante desarrollo.

```
try:
    1 / 0
except Exception as e:
    st.exception(e)
```

- En producción, evita exponer datos sensibles.

MICROFEEDBACK

Toast y globos

Los toasts informan sin interrumpir. Snow y balloons son celebraciones puntuales.

```
st.toast("Tarea guardada")
st.balloons()
```

- No conviertas cada clic en una animacion.

ESPERA

Spinner

Informa que una operación tardara. Colocalo alrededor del trabajo lento.

```
with st.spinner("Cargando..."):
    datos = cargar()
```

- Si conoces avance real, usa progress en vez de spinner.



ACCIÓN **st.button**

Dispara una acción puntual. `type='primary'` destaca la acción principal.

```
if st.button("Calcular", type="primary"):
    st.write(2 + 2)
```

- Deja un solo boton primario por zona.

ENLACE **st.link_button**

Se ve como boton, pero navega a una URL. No ejecuta lógica Python.

```
st.link_button("Abrir guia",
               "https://docs.streamlit.io")
```

- Usa un texto que anticipe el destino.

NAVEGACIÓN **st.page_link**

Crea enlaces internos entre páginas de una app multipágina.

```
st.page_link("pages/datos.py",
             label="Ver datos", icon="📄")
```

- Para estructura completa, usa `st.navigation`.

EVENTO **Boton con callback**

`on_click` ejecuta una función antes de continuar el rerun.

```
def sumar():
    st.session_state.n += 1
st.button("+1", on_click=sumar)
```

- Pasa parámetros con `args=` o `kwargs=`.



WIDGET **st.text_input**

Entrada de una línea. Puede incluir valor inicial, placeholder, ayuda y límite.

```
nombre = st.text_input(
    "Nombre", max_chars=30,
    placeholder="Escribi aqui")
```

- El placeholder no reemplaza una etiqueta visible.

WIDGET **Contraseña**

`type='password'` oculta visualmente el contenido, pero no sustituye una autenticación segura.

```
clave = st.text_input(
    "Clave", type="password")
```

- Nunca muestres ni guardes contraseñas en texto plano.

WIDGET **st.text_area**

Para respuestas largas, comentarios o textos de varias líneas.

```
comentario = st.text_area(
    "Comentario", height=140)
```

- Valida longitud y contenido después de recibirlo.

PATRON **Validación**

Primero recibe, luego limpia y valida. Muestra el error junto al campo.

```
email = st.text_input("Email")
if email and "@" not in email:
    st.error("Falta @")
```

- No uses solo color: agrega texto explicativo.



NÚMERO `st.number_input`

Control preciso para enteros o decimales con límites y pasos.

```
cantidad = st.number_input(
    "Cantidad", min_value=0,
    max_value=100, step=1)
```

- El tipo depende de los valores iniciales y límites.

RANGO `st.slider`

Ideal para explorar rangos. Puede devolver un valor o una pareja.

```
edad = st.slider("Edad", 0, 100, 18)
rango = st.slider("Rango", 0, 10, (2, 8))
```

- No uses slider si se necesita escribir un valor exacto grande.

FECHA `st.date_input`

Selecciona una fecha o intervalo. Python devuelve objetos date.

```
fecha = st.date_input("Fecha")
st.write(fecha.strftime("%d/%m/%Y"))
```

- Define `min_value` y `max_value` cuando corresponda.

HORA `st.time_input`

Selecciona una hora. Combinala con una fecha si necesitas un datetime.

```
hora = st.time_input("Hora")
st.write(hora)
```

- Aclara la zona horaria cuando el contexto sea internacional.



Elegir una opción

Radio, selectbox y control segmentado

STREAMLIT FIELD GUIDE

Edición ampliada 2026

SELECCIÓN **st.radio**

Muestra pocas opciones visibles; el usuario entiende el conjunto completo.

```
tema = st.radio(
    "Tema", ["Claro", "Oscuro"],
    horizontal=True)
```

- Buena opción para 2 a 5 alternativas.

SELECCIÓN **st.selectbox**

Ahorra espacio cuando la lista es mayor. `index=None` permite comenzar sin selección.

```
pais = st.selectbox(
    "Pais", paises, index=None,
    placeholder="Elegir...")
```

- Usa `format_func` para mostrar nombres amigables.

SELECCIÓN **st.segmented_control**

Control compacto para pocas alternativas. Puede permitir una o varias selecciones.

```
vista = st.segmented_control(
    "Vista", ["Lista", "Tarjetas"],
    default="Lista")
```

- Funciona bien para cambiar modo o filtro corto.

ORDEN **st.select_slider**

Elige sobre una secuencia ordenada, incluso si los valores son palabras.

```
nivel = st.select_slider(
    "Nivel", ["Bajo", "Medio", "Alto"])
```

- Las opciones deben tener un orden comprensible.



Elegir varias opciones

Comparar mecanismos de selección múltiple

STREAMLIT FIELD GUIDE

Edición ampliada 2026

MULTIPLE **st.multiselect**

Devuelve una lista. Puede limitar selecciones y mostrar placeholder.

```
tags = st.multiselect(
    "Etiquetas", opciones,
    max_selections=3)
```

- Valida si la lista vacía es aceptable.

MULTIPLE **st.pills**

Opciones visuales compactas con selección simple o múltiple.

```
filtros = st.pills(
    "Filtros", opciones,
    selection_mode="multi")
```

- Útil para categorías breves, no para textos largos.

BOOLEANO **Checkbox**

Representa una decisión independiente de verdadero/falso.

```
acepta = st.checkbox("Acepto")
if acepta:
    st.success("Listo")
```

- No uses checkbox para opciones mutuamente excluyentes.

BOOLEANO **Toggle**

Interruptor para activar o desactivar una configuración inmediata.

```
mostrar = st.toggle("Mostrar detalles")
if mostrar:
    st.write(detalles)
```

- Radio comunica mejor una elección conceptual.



COLOR **st.color_picker**

Devuelve un color hexadecimal seleccionado por el usuario.

```
color = st.color_picker(
    "Color", "#FF4B4B")
st.write(color)
```

- Comprueba contraste si el color se usa en texto.

VALORACION **st.feedback**

Recoge una respuesta visual, por ejemplo estrellas o pulgares.

```
valor = st.feedback("stars")
if valor is not None:
    st.write(valor + 1)
```

- El valor suele ser un índice desde cero.

ACCESIBILIDAD **Etiquetas visibles**

Cada widget necesita una etiqueta clara. `label_visibility` puede ocultarla visualmente sin cambiar el layout.

```
st.text_input("Buscar",
    label_visibility="collapsed")
```

- Si colapsas la etiqueta, aporta contexto accesible alrededor.

AYUDA **disabled y help**

Deshabilita acciones no disponibles y explica por que con `help` o texto cercano.

```
st.button("Enviar",
    disabled=not listo,
    help="Completa los datos")
```

- No dejes al usuario adivinando por que no puede continuar.



ARCHIVO **st.file_uploader**

Recibe uno o varios archivos. Devuelve UploadedFile, similar a un archivo binario.

```
archivo = st.file_uploader(
    "Subir CSV", type=["csv"])
if archivo:
    df = pd.read_csv(archivo)
```

- **Limita extensiones y valida contenido.**

ARCHIVO **Varios archivos**

accept_multiple_files=True devuelve una lista. Procesa cada elemento de forma controlada.

```
archivos = st.file_uploader(
    "Fotos", type=["png", "jpg"],
    accept_multiple_files=True)
```

- **Los límites de tamaño dependen de configuración.**

CÁMARA **st.camera_input**

Permite tomar una foto desde un dispositivo con cámara.

```
foto = st.camera_input("Tomar foto")
if foto:
    st.image(foto)
```

- **El navegador puede solicitar permiso.**

MICROFONO **st.audio_input**

Graba audio desde el navegador y devuelve datos de audio.

```
audio = st.audio_input("Grabar")
if audio:
    st.audio(audio)
```

- **Aclara finalidad, privacidad y almacenamiento.**



FORM Formulario mínimo

Los widgets dentro del formulario no disparan el procesamiento final hasta el submit.

```
with st.form("registro"):
    nombre = st.text_input("Nombre")
    enviar = st.form_submit_button("Enviar")
    if enviar:
        st.success(nombre)
```

- Cada formulario necesita `st.form_submit_button`.

FORM `clear_on_submit`

Limpia los widgets después de enviar. Útil en cargas repetidas.

```
with st.form("alta",
    clear_on_submit=True):
    dato = st.text_input("Dato")
    ok = st.form_submit_button("Guardar")
```

- Guarda primero; luego confirma al usuario.

FORM Validar al enviar

Separa validación, procesamiento y feedback.

```
if enviar:
    if not nombre.strip():
        st.error("Falta nombre")
    else:
        guardar(nombre)
```

- No escribas en base de datos antes de validar.

FORM Límites del form

Solo `st.form_submit_button` puede tener callback dentro del formulario; otros widgets se agrupan.

```
def enviar():
    ...
    st.form_submit_button(
        "Enviar", on_click=enviar)
```

- Los cambios internos no se reflejan afuera hasta enviar.



ESTADO key única

La key identifica un widget y crea una entrada asociada en session_state.

```
nombre = st.text_input(
    "Nombre", key="nombre_usuario")
st.write(st.session_state.nombre_usuario)
```

- Nunca repitas la misma key para dos widgets.

EVENTO on_change

Widgets de entrada pueden llamar una función cuando cambia su valor.

```
def normalizar():
    st.session_state.nombre = st.session_state.nombre.strip()
st.text_input("Nombre", key="nombre",
    on_change=normalizar)
```

- El callback corre antes del resto del script.

EVENTO args y kwargs

Pasa datos adicionales al callback sin ejecutarlo en la declaración.

```
def avisar(tipo, msg=""):
    st.toast(f"{tipo}: {msg}")
st.button("OK", on_click=avisar,
    args=("Info",), kwargs={"msg": "Listo"})
```

- Escribe on_click=función, no on_click=función().

ESTADO Widget dinámico

Si cambia la identidad del widget, puede reiniciarse. Mantene key y parámetros estables.

```
st.slider("Nota", 1, 12,
    key="nota_final")
```

- Ocultar y recrear widgets puede afectar su estado.



LAYOUT with `st.container`

Agrupar elementos en un bloque. Puede usar borde y altura.

```
with st.container(border=True):  
    st.subheader("Ficha")  
    st.write("Contenido")
```

- Una altura fija puede crear scroll interno.

PLACEHOLDER `st.empty`

Reserva un lugar y permite reemplazar o limpiar su contenido.

```
lugar = st.empty()  
lugar.info("Procesando")  
lugar.success("Terminado")
```

- Llama `lugar.empty()` para borrar.

LAYOUT Expander

Ocultar detalles opcionales dentro de un panel desplegable.

```
with st.expander("Ver explicacion"):  
    st.write("Detalle")
```

- No escondas la acción principal.

LAYOUT Popover

Abre contenido flotante sin ocupar espacio permanente.

```
with st.popover("Filtros"):  
    categoria = st.selectbox(  
        "Categoria", opciones)
```

- Ideal para controles secundarios compactos.



LAYOUT **st.columns**

Crea columnas iguales o proporcionales. gap controla separación.

```
izq, der = st.columns([2, 1], gap="large")
with izq: st.write("Principal")
with der: st.metric("Total", 42)
```

- En pantallas angostas pueden apilarse.

LAYOUT **Tabs**

Organiza vistas relacionadas. El contenido de todas las tabs puede calcularse en cada rerun.

```
t1, t2 = st.tabs(["Resumen", "Datos"])
with t1: st.write("Resumen")
with t2: st.dataframe(df)
```

- No uses tabs para un flujo que exige orden.

LAYOUT **Sidebar**

Reserva filtros o navegación secundaria fuera del área principal.

```
with st.sidebar:
    st.header("Filtros")
    año = st.selectbox("Año", años)
```

- Mantene en el cuerpo los resultados y acciones centrales.

LAYOUT **Alineación vertical**

Las columnas permiten alinear controles y botones.

```
a, b = st.columns([3, 1],
    vertical_alignment="bottom")
a.text_input("Buscar")
b.button("Ir", use_container_width=True)
```

- use_container_width puede ocupar el ancho disponible.



DATOS **st.dataframe**

Tabla interactiva con orden, búsqueda, tamaño y configuración de columnas.

```
st.dataframe(df,  
             use_container_width=True,  
             hide_index=True)
```

- Para datos estaticos simples también existe **st.table**.

DATOS **st.table**

Tabla estática: muestra todo lo que entra y no ofrece interacción.

```
st.table(df.head(10))
```

- Evítala para miles de filas.

EVENTO **Selección de filas**

`on_select` puede rerunear la app y devolver un evento de selección.

```
evento = st.dataframe(  
    df, on_select="rerun",  
    selection_mode="multi-row")  
filas = evento.selection.rows
```

- Comprueba que la lista tenga elementos.

DATOS **Dimensiones**

`height`, `width` y `row_height` ayudan a adaptar la tabla al contenido.

```
st.dataframe(df,  
             height=360, row_height=34)
```

- No fijas medidas que rompan pantallas pequeñas.



EDICIÓN **st.data_editor**

Devuelve una copia editada. No modifica por sí solo el CSV ni la base de datos.

```
editado = st.data_editor(
    df, num_rows="dynamic",
    hide_index=True)
```

- **Guarda explícitamente cuando el usuario confirme.**

EDICIÓN **Filas dinámicas**

`num_rows='dynamic'` habilita agregar y borrar filas.

```
nuevo = st.data_editor(df,
    num_rows="dynamic")
```

- **Valida filas incompletas antes de persistir.**

EDICIÓN **Columnas deshabilitadas**

`disabled` puede bloquear toda la tabla o columnas específicas.

```
st.data_editor(df,
    disabled=["id", "creado_en"])
```

- **Los identificadores suelen ser de solo lectura.**

PATRÓN **Guardar cambios**

Compara o persiste el resultado solo tras una acción clara.

```
editado = st.data_editor(df)
if st.button("Guardar"):
    editado.to_csv("datos.csv", index=False)
```

- **En nube, el disco puede ser temporal.**



COLUMNA Número y formato

Define etiqueta, ayuda, formato, límites y paso.

```
st.dataframe(df, column_config={
    "precio": st.column_config.NumberColumn(
        "Precio", format="$ %.2f")
})
```

- El formato visual no cambia el valor real.

COLUMNA Progreso

ProgressColumn convierte números en barras comparables.

```
"avance": st.column_config.ProgressColumn(
    "Avance", min_value=0, max_value=100)
```

- Incluye también el valor numerico cuando aporte precision.

COLUMNA Enlaces e imágenes

LinkColumn e ImageColumn representan URLs con un componente adecuado.

```
"web": st.column_config.LinkColumn("Sitio"),
"foto": st.column_config.ImageColumn("Foto")
```

- No confíes en URLs recibidas sin validación.

COLUMNA Opciones

SelectboxColumn limita valores durante la edición.

```
"estado": st.column_config.SelectboxColumn(
    "Estado", options=["Pendiente", "Listo"])
```

- Evita categorías casi duplicadas.



KPI **st.metric**

Muestra valor principal, variación y contexto. Útil para dashboards.

```
st.metric("Ventas", "$ 12.400",  
delta="8%", border=True)
```

- **Aclara período y unidad.**

KPI **Métricas en fila**

Combina métricas con columnas para una lectura comparativa.

```
a, b, c = st.columns(3)  
a.metric("A", 10)  
b.metric("B", 20)  
c.metric("C", 30)
```

- **Mantene la misma escala cuando se comparan.**

OBJETO **st.json**

Explora diccionarios y listas anidadas con formato plegable.

```
st.json({"usuario": {"nombre": "Ana"}})
```

- **Buena herramienta de depuración para respuestas API.**

STREAM **st.write_stream**

Muestra un generador o stream por partes, útil para texto progresivo.

```
def tokens():  
    yield "Hola "  
    yield "mundo"  
st.write_stream(tokens())
```

- **El tipo de fragmentos determina como se renderizan.**



CHART Líneas y areas

Aceptan DataFrames y permiten indicar ejes y color.

```
st.line_chart(df, x="fecha", y="ventas")
st.area_chart(df, x="fecha", y="ventas")
```

- Líneas para tendencias; areas para magnitud acumulada.

CHART Barras

Compara categorías. Ordena cuando ayude a leer.

```
st.bar_chart(df, x="producto", y="ventas")
```

- Usa barras horizontales si las etiquetas son largas.

CHART Scatter

Explora relacion entre variables; size y color agregan dimensiones.

```
st.scatter_chart(df,
x="altura", y="peso",
color="grupo", size="edad")
```

- Correlacion visual no implica causalidad.

CHART Mapas

st.map ubica puntos usando columnas de latitud y longitud.

```
st.map(df, latitude="lat",
longitude="lon", size="cantidad")
```

- No publiques ubicaciones sensibles.



CHART Altair

Gramatica declarativa: datos, marcas y codificaciones.

```
chart = alt.Chart(df).mark_bar().encode(
    x="categoria:N", y="ventas:Q", color="grupo:N")
st.altair_chart(chart, use_container_width=True)
```

- N nominal; Q cuantitativo; T temporal; O ordinal.

CHART Plotly

Gráficos interactivos con hover, zoom y muchas familias.

```
fig = px.scatter(df, x="x", y="y", color="grupo")
st.plotly_chart(fig, use_container_width=True)
```

- Reduce puntos o agrega si el gráfico se vuelve lento.

EVENTO Selección en gráficos

Algunos gráficos pueden devolver eventos al seleccionar puntos.

```
evento = st.plotly_chart(
    fig, on_select="rerun")
puntos = evento.selection.points
```

- Trata la selección vacía como caso normal.

DISEÑO Principio de eleccion

Nativo: rapidez.

Altair: visualizacion declarativa.

Plotly: interacción y variedad.

- El mejor gráfico es el que responde la pregunta con menos esfuerzo.



MEDIA **st.image**

Muestra ruta, URL, bytes o imagen. Ajusta ancho y agrega caption.

```
st.image("foto.png",  
caption="Descripcion",  
use_container_width=True)
```

- La caption no sustituye una descripción accesible completa.

MEDIA **st.audio**

Reproduce audio desde archivo, bytes o URL compatible.

```
st.audio("audio.mp3")
```

- Ofrece transcripcion cuando el contenido sea educativo.

MEDIA **st.video**

Inserta video y permite indicar subtítulos cuando estan disponibles.

```
st.video("video.mp4",  
subtitles="subtitulos.vtt")
```

- No hagas depender toda la explicación del audio.

MEDIA **PDF embebido**

No hay un visor PDF universal simple en st.*; puedes ofrecer descarga o incrustar con HTML/componentes.

```
st.download_button(  
"Descargar PDF", data=pdf_bytes,  
file_name="guia.pdf", mime="application/pdf")
```

- La descarga suele ser la opción más robusta.



Descargas y exportacion

Entregar resultados con nombre y tipo correctos

STREAMLIT FIELD GUIDE

Edición ampliada 2026

DOWNLOAD Texto

data puede ser texto o bytes. file_name y mime mejoran la experiencia.

```
st.download_button(
    "Descargar", data=texto,
    file_name="resultado.txt",
    mime="text/plain")
```

- Genera el contenido antes o dentro de un fragmento si es costoso.

DOWNLOAD CSV

Convierte DataFrame sin índice y codifica en UTF-8.

```
csv = df.to_csv(index=False).encode("utf-8")
st.download_button("CSV", csv,
    "datos.csv", "text/csv")
```

- Para Excel usa BytesIO y openpyxl/xlsxwriter.

DOWNLOAD JSON

Serializa con ensure_ascii=False para conservar caracteres.

```
texto = json.dumps(datos,
    ensure_ascii=False, indent=2)
st.download_button("JSON", texto,
    "datos.json", "application/json")
```

- Solo serializa tipos compatibles.

DOWNLOAD Preparacion cacheada

Si exportar cuesta, cachea una función pura que devuelva bytes.

```
@st.cache_data
def a_csv(df):
    return df.to_csv(index=False).encode("utf-8")
```

- No caches datos privados sin entender el alcance.



ESTADO Inicializar

Crea la clave solo si no existe para evitar reiniciarla en cada rerun.

```
if "puntos" not in st.session_state:  
    st.session_state.puntos = 0
```

- Inicializa todas las claves cerca del comienzo.

ESTADO Leer y escribir

Puedes usar notacion de atributo o diccionario.

```
st.session_state.puntos += 1  
st.write(st.session_state["puntos"])
```

- No confundas `session_state` con almacenamiento permanente.

ESTADO Estado por usuario

Cada sesión de navegador mantiene su propio estado; puede perderse al recargar o cortar la conexión.

```
st.session_state.usuario = "Ana"
```

- Para persistencia real usa archivo, base de datos o servicio externo.

ESTADO Borrar

Elimina una clave o recorre todas para reiniciar la sesión.

```
del st.session_state["puntos"]  
# o:  
for k in list(st.session_state):  
    del st.session_state[k]
```

- Confirma antes de borrar progreso importante.



PATRON Contador

Un callback modifica la memoria; la interfaz muestra el valor actualizado.

```
if "n" not in st.session_state: st.session_state.n = 0
def inc(): st.session_state.n += 1
st.button("+1", on_click=inc)
st.metric("Puntos", st.session_state.n)
```

- Evita modificar la misma clave después del widget asociado.

PATRON Wizard por pasos

Guarda el número de paso y renderiza solo la pantalla correspondiente.

```
if "paso" not in st.session_state: st.session_state.paso = 1
if st.session_state.paso == 1:
    st.write("Paso 1")
```

- Valida cada paso antes de avanzar.

PATRON Lista temporal

Guarda elementos agregados durante la sesión.

```
if "tareas" not in st.session_state: st.session_state.tareas = []
if st.button("Agregar"):
    st.session_state.tareas.append(tarea)
```

- Copia o persiste si necesitas conservar tras cerrar.

PATRON Evitar alias mutables

Cuando trabajas con listas o DataFrames, entiende si modificas el mismo objeto o una copia.

```
nuevo_df = st.session_state.df.copy()
nuevo_df.loc[0, "x"] = 1
```

- Los objetos mutables compartidos pueden producir sorpresas.



FLUJO **st.stop**

Detiene el rerun actual. Ideal tras una validación fallida.

```
if archivo is None:
    st.info("Subi un archivo")
    st.stop()
st.dataframe(pd.read_csv(archivo))
```

- **Deja un mensaje antes de detener.**

FLUJO **st.rerun**

Interrumpe el rerun actual y comienza otro. Usalo cuando el estado exige refresco inmediato.

```
st.session_state.modo = "lista"
st.rerun()
```

- **Un uso incorrecto puede crear bucles infinitos.**

NAVEGACIÓN **st.switch_page**

Cambia programaticamente a otra página definida en la app.

```
if st.button("Continuar"):
    st.switch_page("pages/resumen.py")
```

- **Valida antes de navegar.**

ORDEN **Callback antes del rerun**

El callback se ejecuta primero; después Streamlit vuelve a correr el script con el nuevo estado.

```
def guardar():
    st.session_state.ok = True
st.button("Guardar", on_click=guardar)
```

- **Este orden explica muchos comportamientos inesperados.**



CACHE `st.cache_data`

Para datos serializables: DataFrames, listas, calculos, transformaciones y respuestas.

```
@st.cache_data(ttl=600)
def cargar(url):
    return pd.read_csv(url)
```

- Cada llamada recibe una copia; ttl limita antigüedad.

CACHE `st.cache_resource`

Para recursos compartidos: modelos, clientes o conexiones costosas.

```
@st.cache_resource
def modelo():
    return cargar_modelo()
```

- El objeto puede compartirse entre usuarios: debe ser seguro.

CACHE Parámetros y hash

La cache distingue argumentos. Un parámetro con nombre iniciado en `_` puede excluirse del hash.

```
@st.cache_data
def transformar(df, _logger=None):
    return df.copy()
```

- No excluyas parámetros que cambian el resultado.

CACHE Limpiar y medir

Puedes borrar la cache de una función o desde la interfaz de la app.

```
cargar.clear()
st.cache_data.clear()
```

- Primero mide el cuello de botella; después cachea.



FRAGMENT @st.fragment

La interacción interna puede rerunear solo el fragmento, no todo el script.

```
@st.fragment
def controles():
    x = st.slider("X", 0, 10)
    st.write(x)
    controles()
```

- Los widgets del fragmento deben vivir en su cuerpo.

FRAGMENT Ejecución periodica

run_every permite actualizar automáticamente una zona mientras la sesión esta activa.

```
@st.fragment(run_every="10s")
def reloj():
    st.write(datetime.now())
```

- No uses intervalos agresivos sin necesidad.

FRAGMENT Rerun completo desde fragment

Cuando un cambio afecta toda la app, solicita un rerun completo.

```
st.rerun(scope="app")
```

- scope='fragment' solo es válido durante un rerun de fragmento.

FRAGMENT Evitar acumulacion

Si escribes fuera del cuerpo principal del fragmento, algunos elementos pueden acumularse.

```
zona = st.container()
@st.fragment
def bloque():
    with zona: st.write("Actual")
```

- Mantene el alcance visual bien definido.



MODAL @st.dialog

Convierte una función en dialogo. La interacción interna puede rerunear solo el dialogo.

```
@st.dialog("Confirmar")
def confirmar():
    if st.button("Si"):
        borrar()
    confirmar()
```

- Usa dialogo para decisiones breves, no para páginas enteras.

AVANCE st.progress

Actualiza una barra entre 0 y 100 o 0.0 y 1.0.

```
barra = st.progress(0)
for i in range(100):
    barra.progress(i + 1)
```

- No simules avance si no conoces el progreso real.

PROCESO st.status

Agrupar pasos y puede terminar como complete, error o running.

```
with st.status("Procesando") as s:
    st.write("Paso 1")
    s.update(label="Listo", state="complete")
```

- Muestra hitos útiles, no logs interminables.

PROCESO Placeholder vivo

Combina st.empty() con mensajes para reemplazar estados sin apilar contenido.

```
msg = st.empty()
msg.info("Cargando")
# trabajo
msg.success("Listo")
```

- Limpia elementos temporales cuando ya no aportan.



PÁGINAS Punto de entrada

Define páginas, crea la navegación y ejecuta la página seleccionada.

```
inicio = st.Page("inicio.py", title="Inicio", default=True)
datos = st.Page("datos.py", title="Datos")
pg = st.navigation([inicio, datos])
pg.run()
```

- `st.Page` + `st.navigation` es el enfoque moderno y flexible.

PÁGINAS Grupos de páginas

Un diccionario organiza secciones en el menú.

```
pg = st.navigation({
    "General": [inicio],
    "Análisis": [datos, graficos]
})
pg.run()
```

- Los nombres de grupo deben ser breves.

PÁGINAS Página desde función

`st.Page` acepta ruta o callable, útil para páginas pequeñas.

```
def ayuda():
    st.title("Ayuda")
p = st.Page(ayuda, title="Ayuda")
```

- Separa archivos cuando la página crezca.

PÁGINAS Navegación dinámica

Construye la lista según estado, rol o datos disponibles.

```
paginas = [inicio]
if st.session_state.get("admin"):
    paginas.append(admin)
st.navigation(paginas).run()
```

- Ocultar el enlace no reemplaza autorización en el servidor.



URL **st.query_params**

Lee y escribe parámetros de la URL como un diccionario.

```
filtro = st.query_params.get("q", "")
st.query_params["q"] = "python"
```

- **Todo viaja como texto; convierte y valida tipos.**

URL **Parámetros repetidos**

Usa `get_all` para una clave repetida y `from_dict` para reemplazar el conjunto.

```
tags = st.query_params.get_all("tag")
st.query_params.from_dict({"tag": ["a", "b"]})
```

- **No pongas secretos en la URL.**

CONTEXTO **st.context**

Expone información disponible de la sesión y solicitud, como headers, cookies o locale según el entorno.

```
idioma = st.context.locale
st.write(idioma)
```

- **Trata headers y cookies como datos no confiables.**

HTML **st.html**

Inserta HTML o CSS. No ejecuta JavaScript por defecto y exige cuidado con contenido no confiable.

```
st.html("<p class='nota'>Hola</p>")
```

- **Prefiere componentes nativos cuando existan.**



TEMA config.toml

Guarda configuración global en `.streamlit/config.toml`.

```
[theme]
primaryColor="#FF4B4B"
backgroundColor="#FFFFFF"
textColor="#172033"
```

- Reinicia el servidor tras algunos cambios.

TEMA Tema claro y oscuro

Define base y prueba contraste en ambos modos.

```
[theme]
base="light"
font="sans-serif"
```

- No dependas solo del color para comunicar estado.

TEMA Fuentes

La configuración moderna admite familias y pesos según versión y despliegue.

```
[theme]
font="sans-serif"
headingFont="sans-serif"
```

- Prioriza legibilidad y disponibilidad.

TEMA CSS mínimo

Si debes usar CSS, encapsula reglas, documenta el motivo y comprueba futuras versiones.

```
st.markdown("""<style>
.nota {padding: 1rem;}
</style>""", unsafe_allow_html=True)
```

- Los selectores internos de Streamlit pueden cambiar.



SECRETOS **st.secrets**

Lee valores definidos en secrets.toml local o en la plataforma de despliegue.

```
token = st.secrets["api"]["token"]
```

- **Nunca subas secrets.toml al repositorio.**

API **Consumir una API**

Usa timeout, comprueba estado y maneja errores.

```
r = requests.get(URL, timeout=10)
r.raise_for_status()
datos = r.json()
```

- **No asumas que la red o el JSON siempre funcionan.**

CONEXIÓN **st.connection**

Crea conexiones configuradas a datos o servicios y centraliza credenciales.

```
conn = st.connection("sql")
df = conn.query("SELECT * FROM tabla", ttl="10m")
```

- **Usa parámetros; evita concatenar entrada del usuario en SQL.**

SEGURIDAD **Error seguro**

Registra el detalle en servidor y muestra al usuario un mensaje útil sin secretos.

```
try:
    datos = cargar()
except Exception:
    logger.exception("Fallo")
    st.error("No se pudo cargar")
```

- **No muestres tokens, rutas internas ni consultas sensibles.**



SQLITE Crear tabla

Usa context manager y parámetros. SQLite es apropiado para proyectos pequeños locales.

```
with sqlite3.connect("app.db") as con:
    con.execute("CREATE TABLE IF NOT EXISTS tareas (id INTEGER PRIMARY KEY, texto TE..")
```

- En nube el disco local puede no ser persistente.

SQLITE Insertar seguro

Los placeholders evitan inyeccion y problemas de escape.

```
con.execute("INSERT INTO tareas(texto) VALUES (?)", (texto,))
```

- Nunca armes SQL concatenando texto del usuario.

SQLITE Leer a DataFrame

Pandas puede convertir una consulta en DataFrame.

```
with sqlite3.connect("app.db") as con:
    df = pd.read_sql_query("SELECT * FROM tareas", con)
```

- Cachea solo si toleras datos momentaneamente desactualizados.

ARQUITECTURA Escalar

Para multiples usuarios y persistencia real, usa una base administrada y transacciones.

```
conn = st.connection("sql")
df = conn.query("SELECT ...")
```

- Separa acceso a datos de la interfaz.



CHAT **st.chat_message**

Crea un turno visual para user o assistant.

```
with st.chat_message("assistant"):  
    st.write("¿En que ayudo?")
```

- **Mantene roles consistentes.**

CHAT **st.chat_input**

Entrada fija para mensajes. Devuelve texto o None.

```
mensaje = st.chat_input("Escribi...")  
if mensaje:  
    st.write(mensaje)
```

- **Valida longitud y contenido.**

CHAT **Historial**

Guarda mensajes en session_state y vuelve a pintarlos en cada rerun.

```
if "mensajes" not in st.session_state:  
    st.session_state.mensajes = []  
for m in st.session_state.mensajes:  
    with st.chat_message(m["role"]): st.write(m["content"])
```

- **Limita historial para controlar costo y rendimiento.**

CHAT **Streaming**

st.write_stream muestra respuestas por fragmentos.

```
with st.chat_message("assistant"):  
    respuesta = st.write_stream(stream)
```

- **Informa errores y permite reintentar.**



TESTING

Primera prueba

AppTest.from_file carga una app y run ejecuta el primer ciclo.

```
from streamlit.testing.v1 import AppTest
at = AppTest.from_file("app.py")
at.run()
assert not at.exception
```

- Usa pytest para organizar el conjunto.

TESTING

Interactuar

Busca widgets, cambia valor, ejecuta y verifica salida.

```
at.text_input[0].set_value("Ana").run()
assert at.success[0].value == "Hola Ana"
```

- Cada interacción necesita run().

TESTING

Por key

Acceder por key suele ser más estable que depender de posiciones.

```
at.text_input(key="nombre").set_value("Ana").run()
```

- Diseña keys claras también para testear.

TESTING

Que probar

Camino feliz.

Entrada vacía o invalida.

Errores de red/datos.

Estado tras varios clics.

- Prueba comportamiento, no detalles decorativos fragiles.



DEBUG Leer el primer error útil

Mira tipo, mensaje y ultima línea de tu código en la traza.

```
st.exception(e)
```

- **SyntaxError ocurre antes de que la app pueda dibujarse.**

DEBUG Inspeccionar valores

Muestra tipo, longitud y una muestra pequeña.

```
st.write(type(df), df.shape)
st.dataframe(df.head())
st.json(datos)
```

- **Retira salidas de depuracion antes de publicar.**

DEBUG Aislar

Reduce la app al ejemplo mínimo que reproduce el fallo.

```
import streamlit as st
st.write("prueba")
```

- **Agrega componentes de a uno hasta encontrar el responsable.**

DEBUG Logs

Usa logging para información del servidor sin llenar la interfaz.

```
import logging
log = logging.getLogger(__name__)
log.info("Filas: %s", len(df))
```

- **Nunca registres secretos o datos personales innecesarios.**



CLOUD Estructura mínima

Repositorio con app, dependencias y recursos usando rutas relativas.

```
mi_app/  
app.py  
requirements.txt  
datos/ventas.csv
```

- Evita rutas como C:\Usuarios\...

CLOUD Community Cloud

Conecta repositorio, elige archivo principal, versión de Python y secretos.

```
streamlit  
pandas  
plotly
```

- La publicación requiere cuenta y repositorio compatible.

CLOUD Rutas robustas

Construye rutas desde el archivo actual.

```
from pathlib import Path  
BASE = Path(__file__).parent  
df = pd.read_csv(BASE / "datos" / "ventas.csv")
```

- Respeta mayúsculas y minúsculas: Linux las distingue.

CLOUD Diagnóstico de build

Revisa logs, nombre del archivo, versión Python y paquetes faltantes.

```
ModuleNotFoundError -> requirements.txt  
FileNotFoundError -> ruta/recurso
```

- Prueba desde una carpeta limpia antes de desplegar.



ARQUITECTURA

Estructura escalable

Divide por responsabilidad, no por tamaño arbitrario.

```
app.py
pages/
src/
  datos.py
  logica.py
  ui.py
tests/
assets/
```

- Evita un app.py de miles de líneas.

ARQUITECTURA

Funciones puras

La lógica que no depende de st.* es más fácil de probar y reutilizar.

```
def promedio(valores):
    return sum(valores) / len(valores)
```

- Separa cálculo de presentación.

ARQUITECTURA

Capa de datos

Centraliza lectura, validación y persistencia.

```
@st.cache_data
def cargar_ventas(ruta):
    return pd.read_csv(ruta)
```

- Devuelve errores de dominio comprensibles.

ARQUITECTURA

Main claro

Un punto de entrada corto cuenta la historia general de la app.

```
def main():
    configurar()
    datos = cargar()
    mostrar(datos)

if __name__ == "__main__": main()
```

- Nombres descriptivos valen más que comentarios obvios.



PERFORMANCE

Mide primero

Cronometra carga, transformación y gráficos antes de optimizar.

```
inicio = time.perf_counter()
df = cargar()
st.caption(f"{time.perf_counter()-inicio:.2f}s")
```

- Optimiza el tramo realmente lento.

PERFORMANCE

Filtra temprano

Lee solo columnas y filas necesarias; agrega antes de graficar.

```
df = pd.read_csv(ruta, usecols=["fecha", "ventas"])
resumen = df.groupby("fecha")["ventas"].sum()
```

- No envíes millones de filas al navegador.

PERFORMANCE

Cache correcta

Cachea operaciones deterministas y recursos costosos con el decorador adecuado.

```
@st.cache_data(ttl="10m")
def resumen(parametros): ...
```

- Incluye en argumentos todo lo que afecta el resultado.

PERFORMANCE

Interfaz progresiva

Muestra estructura y feedback antes de trabajos lentos; usa forms para agrupar filtros.

```
with st.form("filtros"):
    ...
    aplicar = st.form_submit_button("Aplicar")
```

- Evita recalcular por cada pequeño cambio.



SEGURIDAD

Entrada no confiable

Valida tipo, tamaño, rango y formato de todo lo recibido.

```
if archivo.size > 5_000_000:
    st.error("Archivo demasiado grande")
    st.stop()
```

- La extensión no prueba el contenido real.

SEGURIDAD

Secretos fuera del código

Usa `st.secrets` y `.gitignore`. Rota una clave si se publica por accidente.

```
.streamlit/secrets.toml
.env
*.db
```

- Borrar un commit no siempre elimina un secreto del historial.

SEGURIDAD

Autorización

Autenticar dice quien es; autorizar decide que puede hacer.

```
if not usuario.puede("borrar"):
    st.error("Sin permiso")
    st.stop()
```

- Ocultar un botón no protege la operación.

PRIVACIDAD

Minimizar datos

Recolecta solo lo necesario, informa finalidad y define cuanto tiempo se conserva.

```
st.caption("Usamos este dato para...")
```

- Evita mostrar datos de una sesión a otra.



A11Y Jerarquía y etiquetas

Usa título, encabezados y labels reales en orden coherente.

```
st.title("Panel")
st.header("Filtros")
st.selectbox("Año", años)
```

- No dependas de placeholder como única etiqueta.

A11Y Contraste y color

Texto legible y estados acompañados por palabras o iconos.

```
st.error("No se guardó: falta el nombre")
```

- Prueba tema claro, oscuro y zoom.

A11Y Teclado y foco

Ordena controles siguiendo el flujo natural y evita interacciones innecesariamente complejas.

```
with st.form("datos"):
    ...
```

- Comprueba que se pueda completar sin mouse.

UX Prevención y recuperación

Valida cerca del campo, confirma acciones destructivas y conserva el trabajo posible.

```
if st.button("Borrar"):
    confirmar_borrado()
```

- Un buen error explica que paso y como continuar.



RECETA 1. Cargar

Lee y cachea el archivo.

```
@st.cache_data
def cargar():
    return pd.read_csv("datos.csv")
df = cargar()
```

- **Ajusta tipos después de leer.**

RECETA 2. Filtrar

Normaliza mayúsculas y valores vacíos.

```
q = st.text_input("Buscar")
mask = df["nombre"].str.contains(q, case=False, na=False)
filtrado = df[mask]
```

- **regex=False si el texto debe ser literal.**

RECETA 3. Mostrar

Resume cantidad y muestra columnas útiles.

```
st.metric("Resultados", len(filtrado))
st.dataframe(filtrado, hide_index=True)
```

- **Muestra un estado vacío si no hay coincidencias.**

RECETA 4. Descargar

Exporta exactamente la vista filtrada.

```
csv = filtrado.to_csv(index=False).encode("utf-8")
st.download_button("Descargar", csv, "resultado.csv")
```

- **El nombre del archivo debe describir el contenido.**



RECETA 1. Estado

Inicializa índice y puntaje.

```
if "i" not in st.session_state:
    st.session_state.i = 0
    st.session_state.puntos = 0
```

- Guarda también si ya se respondió.

RECETA 2. Pregunta

Obtiene la pregunta actual y muestra opciones.

```
p = preguntas[st.session_state.i]
respuesta = st.radio(p["texto"], p["opciones"], index=None)
```

- index=None evita una respuesta preseleccionada.

RECETA 3. Corregir

Valida, suma y avanza una sola vez.

```
if st.button("Responder", disabled=respuesta is None):
    if respuesta == p["correcta"]:
        st.session_state.puntos += 1
    st.session_state.i += 1
    st.rerun()
```

- Evita sumar dos veces en reruns posteriores.

RECETA 4. Final

Muestra resultado y ofrece reinicio.

```
if st.session_state.i == len(preguntas):
    st.metric("Puntaje", st.session_state.puntos)
    if st.button("Reiniciar"):
        st.session_state.clear(); st.rerun()
```

- Explica respuestas, no solo puntaje.



RECETA Filtro en sidebar

Elige valores disponibles en los datos.

```
with st.sidebar:
    regiones = st.multiselect("Region", sorted(df.region.unique()))
```

- Define si vacío significa todas o ninguna.

RECETA Aplicar filtro

Construye una mascara clara y comprobable.

```
vista = df.copy()
if regiones:
    vista = vista[vista.region.isin(regiones)]
```

- Muestra filtros activos.

RECETA Resumen

Primero contexto; luego visualizacion.

```
a, b = st.columns(2)
a.metric("Ventas", vista.ventas.sum())
b.metric("Pedidos", len(vista))
st.bar_chart(vista, x="mes", y="ventas")
```

- Usa formatos y unidades coherentes.

RECETA Detalle y exportacion

Tabla para inspeccionar y boton para llevarse el resultado.

```
st.dataframe(vista, hide_index=True)
st.download_button("CSV", vista.to_csv(index=False), "vista.csv")
```

- No exportes columnas sensibles por accidente.



ERROR ModuleNotFoundError

La biblioteca no esta instalada o falta en requirements.txt.

```
python -m pip install nombre_paquete
```

- Comprueba que pip corresponde al mismo Python.

ERROR NameError

Usaste una variable antes de crearla o escribiste distinto el nombre.

```
nombre = "Ana"  
st.write(nombre)
```

- Python distingue mayusculas y minusculas.

ERROR FileNotFoundError

La ruta no existe desde la carpeta donde corre la app.

```
BASE = Path(__file__).parent  
ruta = BASE / "datos" / "archivo.csv"
```

- Incluye el archivo en el repositorio.

ERROR DuplicateWidgetID

Dos widgets tienen identidad igual. Asigna keys distintas.

```
for i, item in enumerate(items):  
    st.checkbox(item, key=f"item_{i}")
```

- Las keys deben ser estables entre reruns.



ERROR El boton 'se olvida'

button solo es True en el rerun del clic.

```
if "mostrar" not in st.session_state: st.session_state.mostrar = False
if st.button("Mostrar"): st.session_state.mostrar = True
```

- Usa estado para decisiones persistentes.

ERROR La app recalcula todo

Es el modelo normal de rerun. Agrupa con forms, cachea o usa fragments.

```
@st.cache_data
def costoso(x): ...
```

- No conviertas todo en estado sin necesidad.

ERROR El CSV no se guarda

data_editor devuelve datos editados, pero no persiste automáticamente.

```
editado = st.data_editor(df)
if st.button("Guardar"):
    editado.to_csv(ruta, index=False)
```

- En hosting, usa almacenamiento persistente.

ERROR Funciona local, falla online

Suelen ser rutas, mayusculas, secretos, versión Python o dependencia faltante.

```
requirements.txt
.streamlit/config.toml
assets/
```

- Lee los logs de despliegue desde el primer error.



DECISIÓN **Mostrar**

Texto variado -> `st.write`
Código -> `st.code`
Datos -> `st.dataframe`
Objeto anidado -> `st.json`

- **Elige el componente más semántico.**

DECISIÓN **Elegir**

2-5 opciones visibles -> `radio`
Lista larga -> `selectbox`
Varias -> `multiselect/pills`
Sí/no -> `checkbox/toggle`

- **La cantidad y relación entre opciones decide.**

DECISIÓN **Organizar**

Filtros -> `sidebar`
Comparar -> `columns`
Vistas pares -> `tabs`
Detalle opcional -> `expander/popover`

- **No escondas la acción principal.**

DECISIÓN **Recordar y acelerar**

Sesión -> `session_state`
Datos calculados -> `cache_data`
Recurso global -> `cache_resource`
Solo una zona -> `fragment`

- **Estado y cache resuelven problemas distintos.**

Checklist antes de entregar



Revision funcional, visual y técnica

STREAMLIT FIELD GUIDE

Edición ampliada 2026

CHECK Funcional

Camino principal funciona.
Vacíos y errores están contemplados.
No hay doble envío.
Estado se reinicia correctamente.

● **Prueba como si fueras otra persona.**

CHECK Visual

Jerarquía clara.
Unidades visibles.
No hay scroll innecesario.
Tema claro/oscuro legible.

● **Revisa también en ventana angosta.**

CHECK Técnico

requirements completo.
Rutas relativas.
Sin secretos en Git.
Logs sin errores.

● **Clona en limpio y ejecuta.**

CHECK Comunicacion

README explica uso.
Fuentes citadas.
Limitaciones declaradas.
Datos personales protegidos.

● **Una app profesional también documenta decisiones.**



Índice API - Texto y estado visual

Nombre, intencion y página tematica para ampliar

STREAMLIT FIELD GUIDE

Edición ampliada 2026

- **st.title**
título principal

- **st.header**
encabezado de sección

- **st.subheader**
subseccion

- **st.markdown**
texto con formato

- **st.write**
salida flexible

- **st.text**
texto literal

- **st.caption**
texto secundario

- **st.code**
bloque de código

- **st.latex**
fórmula

- **st.divider**
separador

- **st.badge**
etiqueta compacta

- **st.success/info/warning/error**
feedback

- **st.exception**
traza

- **st.toast**
aviso breve

- **st.balloons / st.snow**
celebracion

- **st.help**
documentacion de objeto



- **st.button**
acción puntual
- **st.download_button**
descargar
- **st.link_button**
enlace externo
- **st.page_link**
enlace interno
- **st.checkbox**
booleano
- **st.toggle**
interruptor
- **st.radio**
una opción visible
- **st.selectbox**
una opción compacta

- **st.multiselect**
varias opciones
- **st.pills**
opciones tipo ficha
- **st.segmented_control**
modo compacto
- **st.select_slider**
escala ordenada
- **st.slider**
número o rango
- **st.number_input**
número preciso
- **st.text_input**
una línea
- **st.text_area**
varias líneas

- **st.date_input**
fecha/rango
- **st.time_input**
hora
- **st.color_picker**
color
- **st.feedback**
valoracion
- **st.file_uploader**
archivo
- **st.camera_input**
foto
- **st.audio_input**
grabacion
- **st.data_editor**
editar tabla



Índice API - Layout y navegación

Nombre, intencion y página tematica para ampliar

STREAMLIT FIELD GUIDE

Edición ampliada 2026

- **st.sidebar**
barra lateral
- **st.columns**
columnas
- **st.tabs**
pestañas
- **st.container**
grupo
- **st.empty**
placeholder
- **st.expander**
detalle plegable

- **st.popover**
panel flotante
- **st.dialog**
ventana modal
- **st.form**
lote de entradas
- **st.form_submit_button**
enviar form
- **st.Page**
definir página
- **st.navigation**
menu y router

- **st.switch_page**
navegar
- **st.set_page_config**
configurar página
- **st.logo**
marca
- **st.html**
HTML/CSS



● **st.dataframe**
tabla interactiva

● **st.table**
tabla estática

● **st.json**
objeto anidado

● **st.metric**
KPI

● **st.line_chart**
líneas

● **st.area_chart**
areas

● **st.bar_chart**
barras

● **st.scatter_chart**
dispersion

● **st.map**
mapa

● **st.altair_chart**
Altair

● **st.plotly_chart**
Plotly

● **st.pydeck_chart**
mapas avanzados

● **st.pyplot**
Matplotlib

● **st.vega_lite_chart**
Vega-Lite

● **st.graphviz_chart**
grafos

● **st.bokeh_chart**
Bokeh



Nombre, intención y página temática para ampliar

- **st.image**
imagen
- **st.audio**
audio
- **st.video**
video
- **st.write_stream**
salida progresiva
- **st.spinner**
espera
- **st.status**
pasos
- **st.progress**
avance

- **st.stop**
detener
- **st.rerun**
repetir
- **st.fragment**
rerun parcial
- **st.session_state**
memoria
- **st.query_params**
URL
- **st.context**
sesión/solicitud
- **st.cache_data**
datos

- **st.cache_resource**
recurso
- **st.connection**
servicio de datos
- **st.secrets**
credenciales
- **st.chat_input**
entrada chat
- **st.chat_message**
turno chat
- **st.components.v1.html**
componente HTML



Fuentes oficiales y criterio de actualización

Consulta la documentación al usar una API reciente o al cambiar de versión

STREAMLIT FIELD GUIDE

Edición ampliada 2026

API reference

<https://docs.streamlit.io/develop/api-reference>

Session State

https://docs.streamlit.io/develop/api-reference/caching-and-state/st.session_state

Cheat sheet oficial

<https://docs.streamlit.io/develop/quick-reference/cheat-sheet>

Caching

<https://docs.streamlit.io/develop/concepts/architecture/caching>

Release notes

<https://docs.streamlit.io/develop/quick-reference/release-notes>

Multipage apps

<https://docs.streamlit.io/develop/concepts/multipage-apps/overview>

Widgets

<https://docs.streamlit.io/develop/api-reference/widgets>

App testing

<https://docs.streamlit.io/develop/concepts/app-testing>

Layouts

<https://docs.streamlit.io/develop/api-reference/layout>

Community Cloud

<https://docs.streamlit.io/deploy/streamlit-community-cloud>

Dataframes

<https://docs.streamlit.io/develop/concepts/design/dataframes>

Configuration

<https://docs.streamlit.io/develop/api-reference/configuration>

Regla de oro

Si una función no existe en tu entorno, comprueba la versión instalada con: `python -m pip show streamlit`